Tokenization + Bengio’s NLPM

P1)

1) Loaded the file’s sentences this is done just using \n as sep total sentences were: 6714.

2) Preprocessor: used NFKC to normalize the sentence, then separated the punctuation with a space, finally joined back the words and removed any extra spaces.

3) Trainer:

Starts as : hinditext = ["he was there .",]
split-
=>["he", "was", "there", "."]
counter becomes-

```
{
  "he": 1,
  "was": 1,
  "there": 1,
  ".": 1
}
```

now using regex libraries grapheme clustering i am handling matras
*case of treating matra alone is also taken (commented out)

chars = ["h", "e"]
result = ["h", "##e"]

splits = {"he": ["h", "##e"],"was": ["w", "##a", "##s"],}

finally,
vocab_set = {"h","##e","w", "##a","##s",}

Now from the global words dict i iterate in all the words and use the split dict to get the split for this word now in this split i update count of single token and adjacent pairs

likelihood=(count of pair)/((count of token 1)*(count of token 2))
tie break=lexicographically smaller pair

Now for words in the splits keys i iterate in the split key->value is the split now if i find a adj pair to be best match i merge them and remove the individual ones by simply skipping the indices.

4) Encoder:
Preprocessed the raw text using part 1
Added bos now in one single sentence now in one single sentence's word split it on graphemes
Now starting from max possible substr length that is full word i start from back and break the first time i find a matching vocab
Added eos and added this encoded list to list of lists(holds all sentences)

Encoding for token id is same but using the vocab to id mapping done in train part i assign a id.

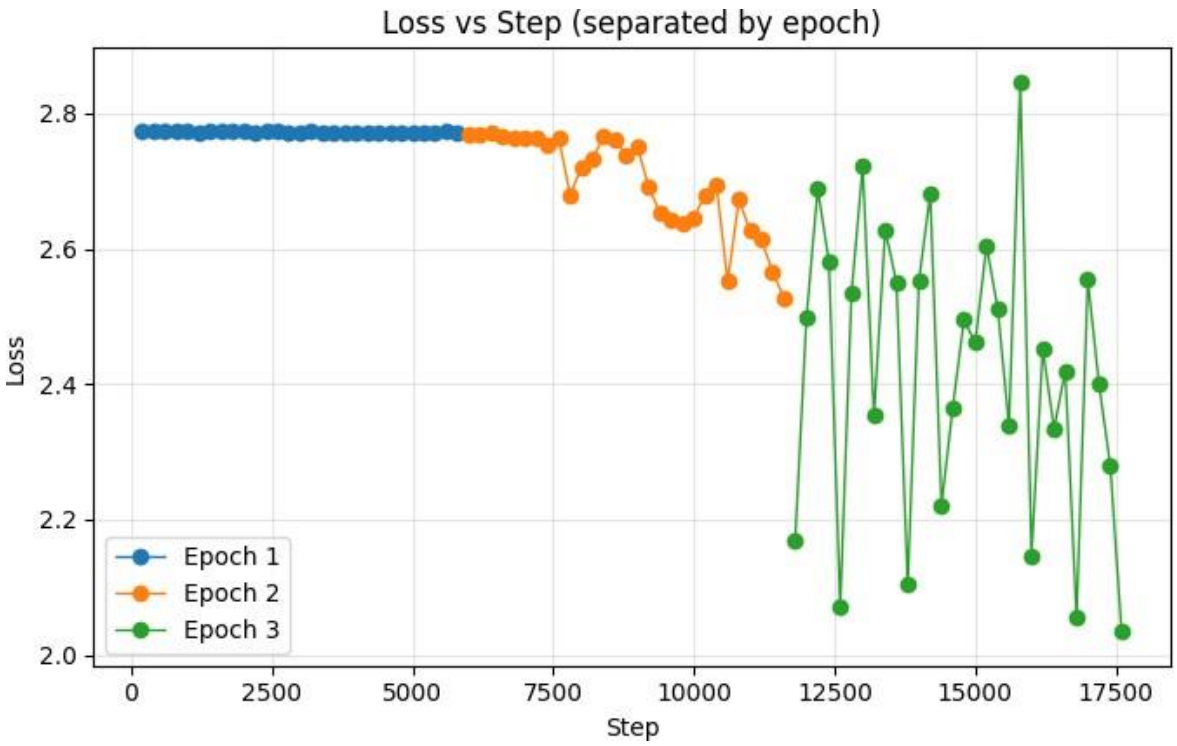
5) Decoder:
Firstly i assigned a id to each token from the vocab file this file was generated by trainer part
Now i have a token list of lists these came from encoder part i read them and make a list of lists by splitting them
Now the main decode part where i iterate in the list we made and using the map made earlier we map token id to token to form a sentence
Finally wrote to a file.

P2) We have implemented Q2) FastText Word Representation as follows =>

A. The corpus given to us is in a text file. We loaded it and then used standard file processing techniques in python.
B. The next and most important step is Normalization + Tokenization. Since the corpus is in Hindi, NKFC normalization is a very important step because it normalizes different unicode variants into standard notation which is very relevant for Hindi language. However, there is no need for us to standard alphabet cases as Hindi language has no concept of capital and small letters hence making “Hello” as “hello” is completely irrelevant. We separated punctuation also using spaces to treat them as different and valid words and removed invisible words -> Hindi language often contains format/control words which are invisible and when we tokenise can show up and hinder our process. The last step is to form tokens on white spaces.
C. Now, the entire point of Q2 is that rather than representing a single token as a single entity we can treat it as an aggregate of its corresponding ngrams. Now the question already tells us the context window (3 to 5) i.e. one word is an amalgamation of its 3grams, 4grams and 5grams. So, we implement this by using simple for loops.
D. Now we get corresponding ngrams for the entire corpus and create a vocabulary by only selecting the unique ones and dropping the duplicate ones. Store them in a dictionary and in lexicographical order and give them unique numbers.
E. Now => each word is represented as a list of numbers associated with each of its ngrams.
F. Now, the main crux of the problem is tackled here => to make every associated list of numbers for each word we do a simple training.
   We select a context window suppose we have 5 words
   a b c d e
   Context Window = 2
   Center Word = c (ie the word for which we have to convert the list of numbers to a meaningful embedding which contains some information)
   Now what the context window tells us is that words which are at a distance of 2 share/are correlated to the center word so the dot product of their embedding must be high. So we consider (b,c), (a,c), (d,c) and (e,c) as positive pairs and their dot product should be high, so to make sense of the training we also use negative pairs. These words are chosen through a probability distribution of words which occur frequently or at a distance context pair (Milkov sampling. In our code sample_k tells us for each positive pair how many negative pairs should be there.

G. Now we run a simple trainer where we fix a learning rate and through a loss function update each corresponding ngram vector to inc product between positive words and reduce between negative pairs.
H. Finally, we end up with actual meaningful ngram vectors and the embedding of the word is the sum of all its corresponding meaningful ngram vectors.
I. The main advantage of using an ngram method is that => Suppose there is some word that is not present in the word level vocabulary and we still want an embedding for it. What we can do is convert it into its ngrams, now whatever ngram are present in the ngram level vocabulary store them and the embedding of the word becomes the sum of the present context infused embeddings. This also gives an embedding which is similar to words which are similar to its meaning,a very big performance jump.
J. Some Design Choices =>
    a) Dimensions = 102 ie the embedding for each word is a 102x1 column vector. I chose this number because the corpus is not too big and the number of unique words is also not that big so 102 is a decent choice.
    b) Number of Epochs = 3 I choose this number because if number of epochs > 3 then the time to train is a little longer than i would like, The loss value in itself is not the only indicator to check quality of the embedding, we can do the morphemes test and word similarity test to also gauge at the quality.
    c) Pre Processing decisions are mentioned in point a
    d) File Format is text and Json everywhere => this might be a little inefficient but file operations with these types of files are comparatively easier compared to numpy binary files (npy/npz).

Loss Function vs Epoch Plots =>



Qualitative Analysis of OOV =>

What we are basically doing is if the word is in word level vocabulary it will directly go through the dictionary and give the learned 102 row vector embedding which is the sum of all its learned ngram 102 embeddings. Otherwise it will make all the ngram of the word and those present in the ngram level dictionary it will return the sum of those embeddings.

I have attached screenshots of both the cases -

Case 1 (Word Not Present) अतिदु यवहाय

```
The word is not in the vocabulary, Hence it will not have a learned vector.
However we can split the word into its ngrams, and then use ngrams already present to create a representation:
[-0.022476   -0.47746497  0.104758    0.309536    0.4208369   0.6837021
 -0.231596    0.075912    0.182414    0.026569   -0.21676704  0.026199
 -0.11450697  0.19900498 -0.209659   -0.07398002 -0.10709701 -0.024563
  0.23589797 -0.61867696  0.334886    0.29897195  0.254778    0.09464002
  0.117924   -0.112088   -0.049029    0.5161109  -0.39087003  0.24676701
  0.16576904  0.33244997  0.034774   -0.06903599  0.094765   -0.32398197
  0.47866607 -0.030487    0.245409   -0.27586305  0.30624402 -0.044235
 -0.56610507  0.5883411   0.39544892  0.18670602  0.023178   -0.27353403
  0.074268    0.0018     -0.35169005 -0.23586     0.49208    -0.375149
 -1.2535157  -0.004021    0.09614101  0.727589    0.28970194 -0.20391001
 -0.16994798  0.215418    0.21438196 -0.26130503 -0.07830799  0.36186105
 -0.05561101  0.064469    0.43704697 -0.15223199  0.14535297  0.556059
  0.06839701 -0.18083501 -0.37188196 -0.17411298  0.474345   -0.389065
  0.22644101  0.24000603 -1.1152601  -0.22792803  0.8346     -0.12722498
  0.52974904 -0.099741    0.34957102 -0.8177989  -0.5220661   0.17395599
  0.6144601  -0.16038997  0.39842898 -0.185258   -0.106671    0.25905204
 -0.34627494 -0.25755292 -0.061361    0.223972    0.013911   -0.09329499
  0.6430591  -0.252893   -0.322138    0.305189    0.25116897 -0.29238302
  0.125388    0.29833198  0.26263303 -0.62640697  0.485527   -0.081294
  0.071165    0.26099002 -0.12153201 -0.142914    0.11044601 -0.03707499]
```

Case 2 (Word Present) अंत रक्ष

```
The word is in the vocabulary, Hence it will have a learned vector. The vector is as follows :
[-5.47900e-03 -3.17664e-01  5.37150e-02  2.08825e-01  2.97743e-01
  4.65769e-01 -1.62509e-01  5.45310e-02  1.36559e-01  1.39910e-02
 -1.78676e-01  2.24910e-02 -9.91000e-02  1.41361e-01 -1.69454e-01
 -4.38880e-02 -8.66070e-02 -2.87270e-02  1.61850e-01 -4.16152e-01
  2.23206e-01  2.25859e-01  1.60428e-01  5.81370e-02  8.65060e-02
 -7.63580e-02 -8.74000e-04  3.64576e-01 -2.82637e-01  1.97337e-01
  1.15767e-01  2.37694e-01  1.74800e-02 -2.17280e-02  6.11410e-02
 -2.01795e-01  3.23978e-01 -8.99200e-03  1.59022e-01 -1.82409e-01
  2.20717e-01 -2.24550e-02 -3.61717e-01  3.94831e-01  2.84234e-01
  1.13874e-01 -7.51600e-03 -1.81569e-01  4.76800e-02  1.46610e-02
 -2.09112e-01 -1.33242e-01  3.40299e-01 -2.74383e-01 -8.23797e-01
  5.30000e-05  8.38150e-02  5.03549e-01  2.03911e-01 -1.52533e-01
 -1.39476e-01  1.24140e-01  1.26223e-01 -1.75003e-01 -4.58160e-02
  2.53011e-01 -7.21730e-02  4.29810e-02  3.04520e-01 -1.03286e-01
  1.13468e-01  4.07076e-01  5.37930e-02 -1.23321e-01 -2.49881e-01
 -1.30596e-01  3.38843e-01 -2.49415e-01  1.59006e-01  1.33322e-01
 -7.61860e-01 -1.57049e-01  5.56698e-01 -8.07050e-02  3.85119e-01
 -6.64550e-02  2.48402e-01 -5.55822e-01 -3.65996e-01  1.09653e-01
  4.11724e-01 -1.27832e-01  2.74315e-01 -1.22216e-01 -7.05670e-02
  1.55276e-01 -2.54619e-01 -1.67099e-01 -3.21680e-02  1.35421e-01
  1.13340e-02 -8.95400e-02  4.11109e-01 -1.95964e-01 -2.07141e-01
  2.21492e-01  1.73503e-01 -1.99113e-01  8.80770e-02  2.18549e-01
  1.60714e-01 -4.15271e-01  3.08052e-01 -2.99420e-02  5.34370e-02
  1.91232e-01 -1.09771e-01 -9.15630e-02  8.70880e-02 -2.48270e-02]
```

Qualitative Analysis of Morphological Alignment =>

Now, in this section we basically want to validate the idea that representing the word using all its ngrams is correct or not. The crux of it is basically we have the learned word embedding which is the sum of all its ngram embeddings and the learned ngram embeddings.

So what we will do is, to find the importance of the corresponding ngram for that word we will subtract its embedding from the words embedding and then do a cosine similarity of the ngram vector and the resultant word vector. We will then rank them based on the cosine metric to find what all ngrams were important.

Word One - अक्स

```
अक्स
[('्स>', 0.9992784917386716), ('क्स', 0.9997273959606423), ('<अक', 0.9998288206635542), ('क्स>', 0.9998579654137558), ('<अक्स', 0.9998620844923605), ('<अक्स>', 0.9998650079121328), ('अक्', 0.9
998668455327347), ('<अक्', 0.9998681079981718), ('अक्स', 0.9998712766275757), ('अक्स>', 0.9998729124073571)]
```

Word two - अंत रक्ष

```
अंतरिक्ष
[('क्ष', 0.999791323938754 3), ('<अं', 0.9998878782892012), ('िक्', 0.9999258074993799), ('ंतर', 0.9999370328027962), ('क्ष>', 0.9999406996054 7), ('रिक्', 0.9999420652116303), ('िक्ष', 0.999943962888
7072), ('तरिक्', 0.999944722543707), ('तरि', 0.9999447611082778), ('<अंतरिक्ष>', 0.9999470683187165), ('अंत', 0.9999477779749275), ('<अंत', 0.9999483151245109), ('रिक', 0.9999483771899396), ('<अंत
र', 0.99994854559436), ('अंतरि', 0.99995014925 4648), ('रिक्ष', 0.9999502964154813), ('अंतर', 0.9999508863317352), ('ंतरिक', 0.9999511081483942), ('्ष>', 0.9999514705540927), ('तरिक', 0.9999540588
21379), ('ंतरि', 0.9999555150856028), ('िक्ष>', 0.9999556764872126)]
```

We can see the most important ngrams are usually morphemes of the word, hence validating the idea by intuition.

P3) We have implemented Q3) Neural Probabilistic Language Model as follows =>

A. Q1 already gives us a tokenised corpus using Word Piece Tokenizer. This is the input (Key Assumption) - Not rewriting the code for tokenization and everything.
B. Now what we are doing in this question is that -> We have already been given a vocabulary. We have saved the tokenized corpus and saved it on our local drive.
C. Now to encode, we are looking at each token from the corpus and encoding as its index from the vocab file which has already been given to us. If token not present in the given vocabulary replace it with <unk> id
D. These IDs are not "meaningful numbers", they are just labels / indices
E. Now what happens is that these numbers are not the real input, the model self learns ideal embeddings for the words also.
F. Now, how we provide the training data is that for each word we define a context window -
   Suppose we have a sentence a b c d e f
   And say context window is 4
   So what this is trying to say is that the word f is only influenced by words b c d e i.e. words behind it with a distance less than the context window.
   So we randomly select sentences and for each sentence we take a word and its context window words as input output pairs like for any supervised model.
   Then we train it in epochs like a classical machine learning framework.
G. During training, we replace each context token with <unk> with probability 0.01 (Assignment Requirement)
H. Model Architecture => Input is the context window encoding of the words, then there is a lookup table where the encoding id - embedding pair is there and replaces each encoding with its respective embedding. Then we concatenate all of it into a single embedding . Then we have a simple hidden fully connected linear and an activation function to capture non linearity. Further connected by a layer which has as many neurons and number of words in the vocabulary. We then softmax across all output neurons to get probability-like quantities to decide what the next word will be.
I. The model then can predict based on the hyperpaper k how many tokens we want it to generate for a specific input. K is the number of tokens. We then merge them using word piece merger also because initial tokens were also created using this in Q1 to give the final predicted output.

J. Some Design Choices =>

a) We are comparing 3 different models => One differs in the window size and the other differs in the context window

Model a (ctx4_120) => context Window is 4 and Hidden Layers is 120

Model b (ctx6_120) => context Window is 6 and Hidden Layers is 120

Model c (ctx4_240) => context Window is 4 and Hidden Layers is 240

b) concatenation → Linear → Tanh → Linear → Softmax : We have chosen this as the pipeline because this is a standard NLPM and is very common and tested. We could have stuck to another activation function but decided against it because ChatGPT suggested Tanh.

c) Batch Size is large to speed up the process on CUDA as we are working on a kaggle notebook.

d) Adam optimiser because this converged faster on such types of NLPM models.

e) We have tried to stay faithful to the [3] paper mentioned in the assignment in terms of architecture.

f)

K. For decoding we use the standard Workpiece merge rule.

L. Hyperparameters used:

Model Architecture Hyperparameters:

Hyperparameter	Value(s)	Where in Code
Context Size (c)	4, 6	CONFIGS
Embedding Dimension	66	CONFIGS["emb"]
Hidden Layer Size	120, 240	CONFIGS["hid"]
Activation Function	Tanh	self.act = nn.Tanh()
Output Dimension	Vocabulary size	fc2 layer

Training Hyperparameters

Hyperparameter	Value	Where in Code
Optimizer	Adam	optim.Adam()
Learning Rate	0.001	LEARNING_RATE
Batch Size	1024	BATCH_SI/E
Epochs	12	EPOCHS
Loss Function	CrossEntropyLoss	nn.CrossEntropyLoss()
UNK Replacement Probability	0.01	UNK_PROB
Weight Initialization	PyTorch default	implicit
Device	CUDA or CPU	DEVICE

Data / Input Hyperparameters

Hyperparameter	Value	Where in Code
Special Tokens	<s>, </s>, <UNK>	load_vocab()
Padding Token	<s>	make_windows()
Train/Validation Split	90% / 10%	split_train_val()
Context Type	Left context only	make_windows()
Window Construction	Sliding window	make_windows()

Experimental Model Setups

Model	Context Size	Hidden Dimension	Embedding Dimension
ctx4_120	4	120	66
ctx6_120	6	120	66
ctx4_240	4	240	66

Inference Hyperparameters

Hyperparameter	Value	Where in Code
Generation Length (k)	15	generate()
Decoding Strategy	Greedy (argmax)	generate()
Stop Condition	</s> token	generate()

Metrics Reported

Metric	Computation	Where in Code
Cross Entropy	Mean per-token CE	compute_metrics()
Perplexity	exp(CE)	compute_metrics()
Accuracy	Next-token accuracy	compute_metrics()

Training and Validation Loss Curves for all the 3 Models -

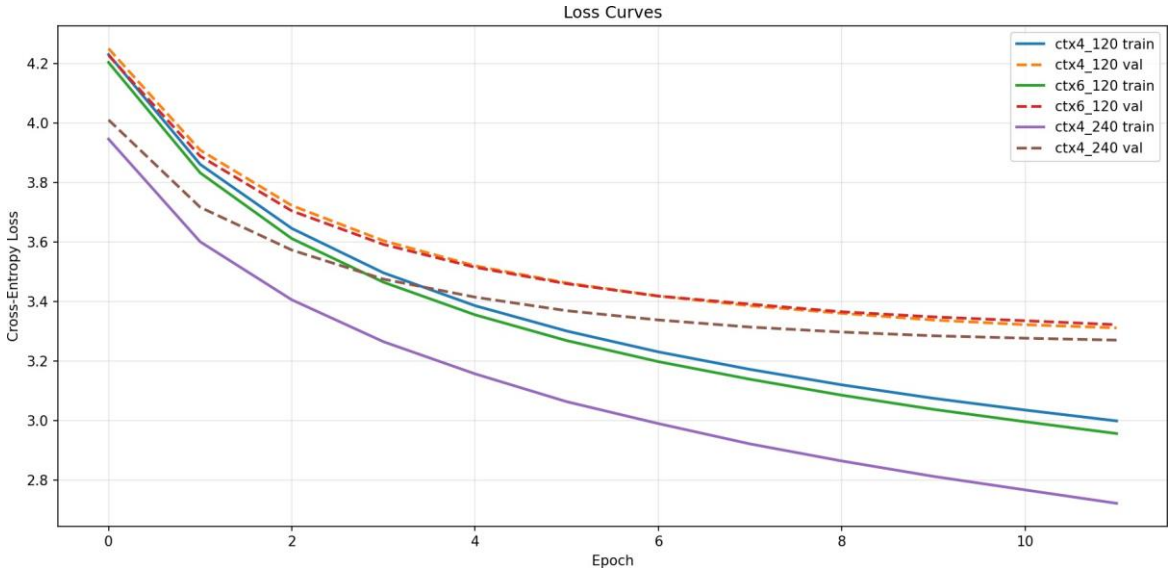


Table Reporting Metrics -

FINAL SUMMARY (last epoch)

Model	Train PPL	Val PPL	Train Acc	Val Acc
ctx4_120	20.06	27.42	0.3841	0.3688
ctx6_120	19.22	27.72	0.3916	0.3651
ctx4_240	15.20	26.32	0.4128	0.3790

Sample Output Generation with generation length 15((tokens) we later merge them -

भारत एक बिशाल ने हेंदिरहा कि <UNK>न <UNK>र के <UNK>र सै <UNK>त क

सरकार ने आजलन के <UNK>न मैं <UNK>र ने <UNK>र को <UNK>र क

खेल के मैदान मैं <UNK>स ने <UNK>र को <UNK>र करने के लिए

मुड़ो लगता है ।

Thank You